

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Trabajo Terminal: 2026-A114

Simulación de sistemas complejos mediante autómatas celulares neuronales

Angeles Lopez Erick Jesse

Director:

Dr. Genaro Juárez Martínez

Contenido

Introducción

Justificación

Objetivos

Estado del arte

Marco metodológico

Metodología

Análisis

Reglas de negocio

Requisitos funcionales

Tecnologías

Arquitectura

Conclusiones

Sistema

Definition

Un sistema es la abstracción matemática de un proceso presente en la naturaleza que tiene sus propias reglas lógicas [1].

Sistemas complejos

Definition

De acuerdo con [2]-[5], un sistema complejo se caracteriza por:

- Están compuestos de muchos elementos.
- Son dinámicos.
- Son sensibles a las condiciones iniciales.
- El comportamiento colectivo es mas que la suma de sus componentes individuales.

Autómata celular

Definition

Siguiendo a [6]-[8], un autómata celular es una 4-tupla $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ donde:

- $\mathcal{L} \subseteq \mathbb{Z}^D$ es el espacio celular D-dimensional.
- $\mathcal{S}$ es un conjunto de estados.
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ es el conjunto de vectores de desplazamiento que definen la vecindad de una célula en $\mathcal{L}$.
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ es la regla local del autómata

Juego de la vida

1. Espacio: bidimensional
2. Estados: Vivo y muerto
3. Vecindad: Moore
4. Función de transición:
   - Si la célula está viva y tiene 2 o 3 vecinos vivos, permanece viva.
   - Si la célula está muerta y tiene 3 vecinos vivos, revive.
   - En cualquier otro caso, la célula muere [9].

Juego de la vida

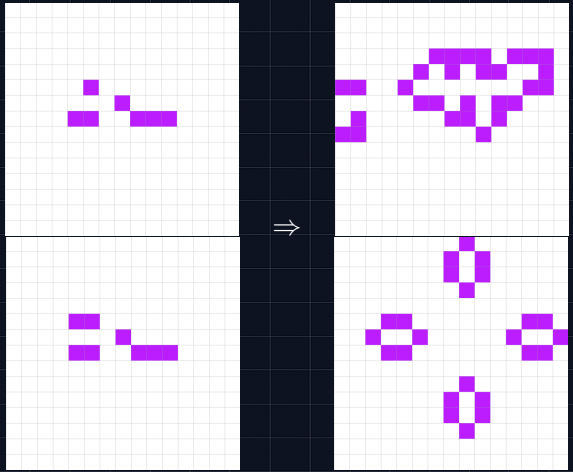


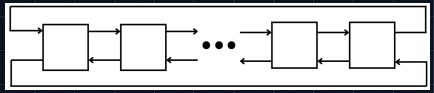
Figura: Evolución de distintos patrones en el Juego de la Vida.

Condición de frontera

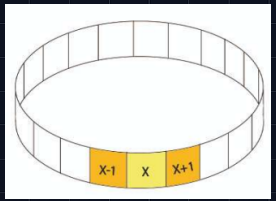
- Espacio finito.
- Condiciones de frontera:



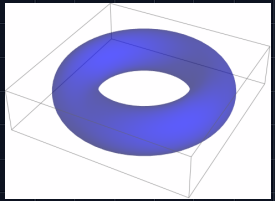
(a) Frontera abierta



(b) Frontera periódica



(a) Frontera periódica unidimensional **anillo**



(b) Frontera periódica bidimensional

Función de transición

La regla mapea $\mathcal{S}^m \to \mathcal{S}$ y puede ser:

- Regla totalística
- Regla semitotalísticas
- Regla arbitraria

Autómata celular elemental

111	110	101	100	011	010	001	000
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$

Cuadro: Cada posible combinación de tres células puede adquirir uno de dos valores

Representación tabular y decimal

Binario	111	110	101	100	011	010	001	000	
Decimal	7	6	5	4	3	2	1	0	**regla**
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	1
					⋮				⋮
	0	0	0	1	1	1	1	0	30
					⋮				⋮
	0	1	1	0	1	1	1	0	110
					⋮				⋮
	1	1	1	1	1	1	1	1	255

Cuadro: Para convertir la regla tabular en la decimal, solo es necesario convertir el número binario.

Representación tabular y decimal

La regla 10 en dos configuraciones diferentes:

Configuración		Representación binaria
$\lvert\mathcal{N}\rvert = 2,\ \lvert\mathcal{S}\rvert = 2$	$\Rightarrow$	1010
$\lvert\mathcal{N}\rvert = 3,\ \lvert\mathcal{S}\rvert = 2$	$\Rightarrow$	000000101

Ejemplo: Propagación del fuego

1. Espacio: bidimensional
2. Estados: Materia viva, materia quemándose y materia quemada
3. Vecindad: Moore
4. Función de transición:
   - Si la materia viva tiene un vecino quemándose, entonces se quema.
   - Si la materia se está quemando, pasa a materia quemada.Si la materia se está quemando, pasa a materia quemada.

Justificación

Limitación actual
El diseño de la función de transición en autómatas celulares depende de expertos y debe hacerse manualmente, lo cual restringe sus aplicaciones prácticas.

Complejidad de los sistemas reales
Muchos fenómenos naturales, sociales o biológicos no poseen reglas explícitas o son demasiado complejos para modelarse de forma analítica.

Justificación

Necesidad
Se requiere un enfoque capaz de aprender comportamientos sin depender de funciones matemáticas definidas manualmente.

Nuevo enfoque
Los autómatas celulares neuronales permiten que la función de transición sea aprendida directamente a partir de datos.

Justificación

Aprovechamiento de IA
Las redes convolucionales y recurrentes pueden capturar dependencias espaciales y temporales, siendo adecuadas para aproximar funciones de transición.

Reducción de complejidad
Permiten simular sistemas complejos sin ajustes manuales, evitando procesos costosos o inviables.

Justificación

Aporte científico
El proyecto integra inteligencia artificial, vida artificial y simulación, creando una plataforma flexible para futuras investigaciones.

Impacto a largo plazo
Se genera un entorno de código abierto, reutilizable y extensible, que puede impulsar nuevas líneas de trabajo en sistemas complejos e IA.

Objetivo

Desarrollar un sistema de simulación basado en autómatas celulares neuronales bidimensionales que permita analizar la generación, mantenimiento, recuperación y propagación de patrones de información en entornos complejos y dinámicos.

Objetivos específicos

Diseñar e implementar un simulador de espacios discretos bidimensionales que sirva de plataforma para estudiar los autómatas celulares neuronales.

Entrenar y evaluar el desempeño de los autómatas celulares neuronales bajo diferentes configuraciones.

Replicar configuraciones clásicas con comportamientos y soluciones conocidas.

Analizar y comparar diferentes comportamientos con base en la red neuronal establecida.

Morfogénesis



Figura: Evolución del *emoji* de una lagartija a partir de una única célula activa después de 219 pasos evolutivos [10].

MNIST



Figura: Clasificación de números [11].

Mesh

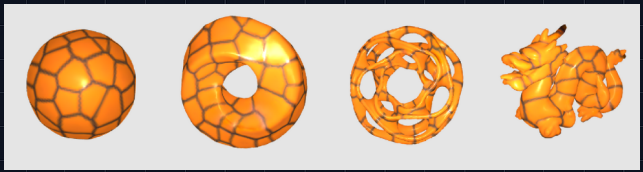


Figura: El entrenamiento se realiza sobre un poliedro simétrico y el autómata celular neuronal puede ser extrapolado a cualquier objeto tridimensional [12].

Población y muestreo

- Autómatas celulares finitos definidos en un espacio bidimensional con una cuadrícula regular.
- Condiciones de frontera.
- Conjunto finito de estados.
- Cualquier función de transición que mapee de $\mathcal{R}^n \rightarrow \mathcal{R}$

Se selecciona un subconjunto mediante un muestreo no probabilístico de tipo intencional.

Investigación

- Tipo aplicada.
- Diseño cuasiexperimental.
- Enfoque cuantitativo.

Procedimiento

- Diseño, desarrollo e implementación del simulador de autómatas celulares.
- Diseño, desarrollo e integración del módulo de red neuronal artificial, encargado de aproximar la función de transición.
- Imitación de patrones conocidos, a fin de validar el desempeño del modelo neuronal frente a sistemas con comportamiento bien caracterizado.
- Simulación de otros sistemas complejos, evaluando la capacidad generalizadora de la aproximación propuesta.

Metodología de desarrollo

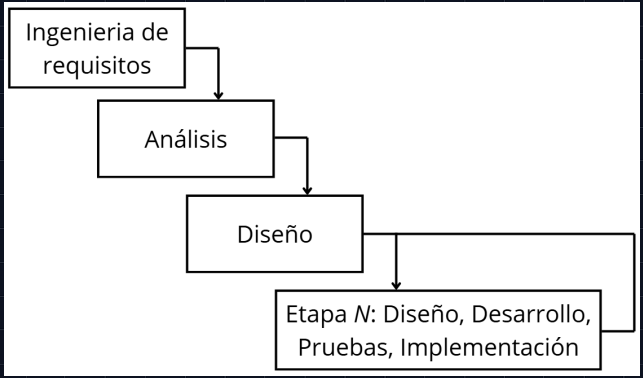


Figura: Modelo de entrega por etapas para el desarrollo del software.

Primer etapa

Simulador global de autómatas celulares.

- Red regular cuadricular con condición de frontera.
- Personalización de estados.
- Personalización de vecindad.
- Definición de función de transición.
- Controles de simulación.

Reglas de negocio

RN1. Para iniciar la simulación es necesario que el conjunto de estados, la topología de la vecindad, la condición de frontera y la función de transición hayan sido definidos.

RN2. Una vez iniciada la simulación, no se puede cambiar el conjunto de estados, la topología de la vecindad ni la función de transición.

RN3. El estado de una celda en el tiempo $t + 1$ depende únicamente de su estado actual y de los estados de sus vecinos.

Reglas de negocio

RN4. El usuario puede manipular el estado de una celda y las condiciones de frontera en tiempo de ejecución.

RN5. El usuario puede pausar, detener, reanudar o retroceder la simulación sin pérdida de información.

RN6. Solo se permite una simulación activa por sesión.

Reglas de negocio

RN7. Los datos importados deben cumplir con el formato definido por el sistema para garantizar su correcta interpretación.

RN8. Los datos exportados deben conservar el mismo formato que los datos importados.

Requisitos funcionales

RF1. El sistema debe mostrar una cuadrícula regular y finita que represente el espacio celular.

RF2. El sistema debe permitir al usuario interactuar con el espacio celular, asignando o modificando los estados de las celdas.

RF3. El sistema debe permitir al usuario definir el conjunto de estados, la topología de la vecindad y las condiciones de frontera.

RF4. El sistema debe permitir la evolución del espacio celular mediante un único paso o de forma continua, ajustando la velocidad de actualización.

Requisitos funcionales

RF5. El sistema debe permitir la importación y exportación de configuraciones y evoluciones del espacio celular.

RF6. El sistema debe permitir al usuario establecer la función de transición manualmente.

RF7. El sistema debe permitir configurar una red neuronal para imitar la evolución de un autómata celular dada una trayectoria objetivo.

RF8. El sistema debe entrenar una red neuronal con los parámetros proporcionados por el usuario para replicar dicha evolución.

Diagrama de casos de uso

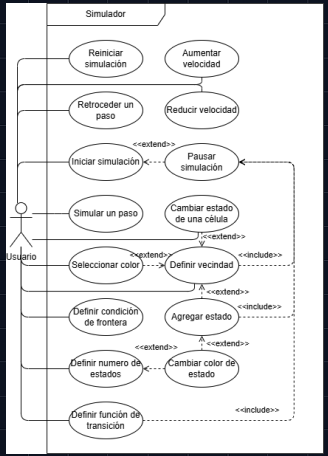


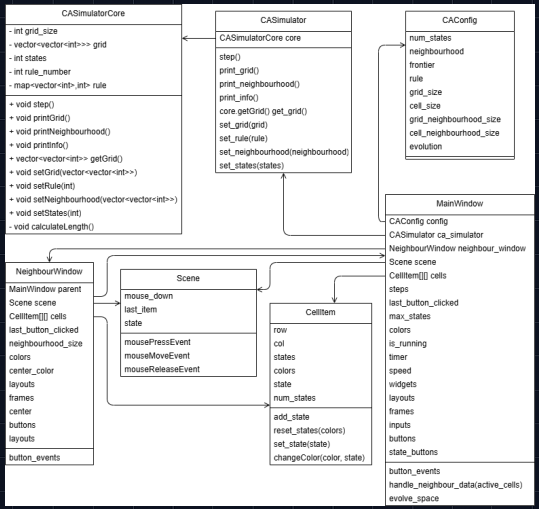
Figura: Diagrama de casos de uso.

Tecnologías



Figura: Tecnologías usadas

Diagrama de clases



Representación del autómata celular neuronal

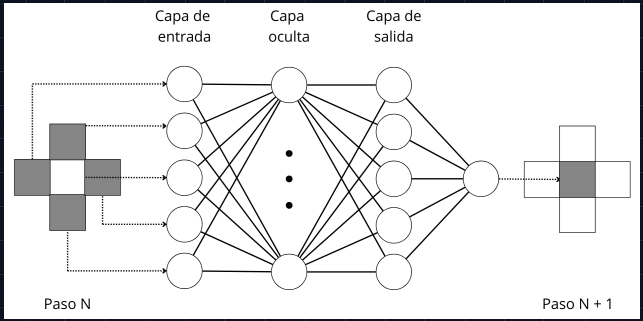


Figura: Representación grafica de la red neuronal.

Arquitectura del simulador

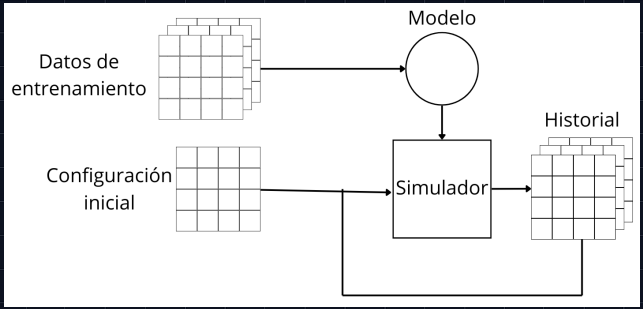


Figura: Arquitectura del simulador.

Interfaz principal

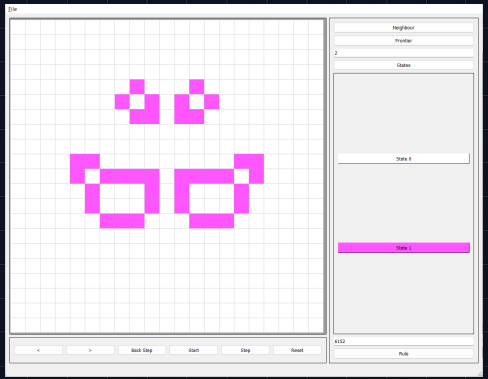


Figura: Interfaz principal.

Interfaz de vecindad

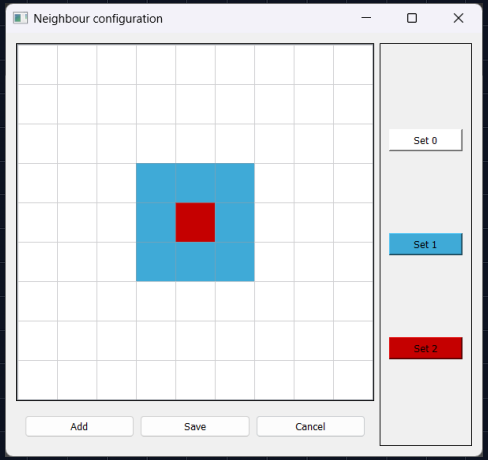


Figura: Interfaz de vecindad.

Conclusiones

Los resultados obtenidos hasta ahora demuestran que el simulador cumple con los requerimientos iniciales y ofrece un entorno estable para las siguientes etapas. Con esta plataforma ya consolidada, el siguiente paso será integrar y evaluar los modelos neuronales seleccionados, lo cual permitirá validar el enfoque propuesto y completar la funcionalidad final del sistema.

¡Gracias!

Referencias I

[1] E. Equipo editorial, "Sistema - Características, tipos y ejemplos", Accedido el 8 de noviembre de 2025, Concepto. (2025), dirección: https://concepto.de/sistema.

[2] I. Zelinka, A. Sanayei, H. Zenil y O. E. Rössler, eds., *How Nature Works*. Heidelberg: Springer International Publishing, 2014, Accedido el 23 de septiembre de 2025. DOI: 10.1007/978-3-319-00254-5. dirección: https://doi.org/10.1007/978-3-319-00254-5.

[3] I. Zelinka y J. Burguillo, "Complex Systems", en ene. de 2018, págs. 11-34, ISBN: 978-3-319-69896-0. DOI: 10.1007/978-3-319-69898-4_2.

[4] S. Moriello, *Sistemas complejos, caos y vida artificial*, https://www.redcientifica.com/doc/doc200303050001.html, Accedido el 23 de septiembre de 2025, n.d.

Referencias II

[5] J. Torres, "Introducción a los sistemas complejos", Accedido el 8 de noviembre de 2025, Universidad de Granada. (2025), dirección: https://www.ugr.es/~jtorres/tema_1_sistemas_complejosv2.pdf.

[6] K. Bhattacharjee, "Cellular Automata: Reversibility, Semi-Reversibility and Randomness", Accedido el 23 de agosto de 2025, Tesis doct., Indian Institute of Engineering Science y Technology, Howrah, West Bengal, 2019.

[7] H. Sayama, *11.1: Definition of Cellular Automata*, https://math.libretexts.org/Bookshelves/Scientific_Computing_Simulations_and_Modeling/Introduction_to_the_Modeling_and_Analysis_of_Complex_Systems_(Sayama)/11%3A_Cellular_Automata_I__Modeling/11.01%3A_Definition_of_Cellular_Automata, Accedido el 24 de septiembre de 2025. Licencia CCBY-NC-SA, n.d.

Referencias III

[8] A. Ilachinski, *Cellular Automata: A Discrete Universe*. Singapore: World Scientific, 2001, Center for Naval Analyses, USA.

[9] Mathematik, Informatik, Naturwissenschaft und Technik am Computer, *John Conway's Game of Life - An Introduction to Cellular Automata*, https://beltoforion.de/en/game_of_life, Accedido el 11 de noviembre de 2025, nov. de 2025.

[10] A. Mordvintsev, E. Randazzo, E. Niklasson y M. Levin, "Growing Neural Cellular Automata", *Distill*, 2020, https://distill.pub/2020/growing-ca. DOI: 10.23915/distill.00023.

[11] E. Randazzo, A. Mordvintsev, E. Niklasson, M. Levin y S. Greydanus, "Self-classifying MNIST Digits", *Distill*, 2020, https://distill.pub/2020/selforg/mnist. DOI: 10.23915/distill.00027.002.

Referencias IV

[12] E. Pajouheshgar, Y. Xu, A. Mordvintsev, E. Niklasson, T. Zhang y S. Süsstrunk, *Mesh Neural Cellular Automata*, 2024. arXiv: 2311.02820 [cs.CV]. dirección: https://arxiv.org/abs/2311.02820.

Riesgos técnicos

RT1 Incompatibilidad entre diferentes lenguajes de programación: los módulos desarrollados en Python, C++ y CUDA generan errores de integración al intercambiar información y sincronizar procesos.

RT2 Indisponibilidad de hardware: La tarjeta gráfica no tiene el rendimiento suficiente.

RT3 Incompatibilidad entre tecnologías: El sistema operativo, los lenguajes de programación y los elementos de hardware no son compatibles para el desarrollo y ejecución.

RT4 Dependencias de librerías: La instalación y configuración de diferentes librerías varía según el sistema operativo.

RT5 Mala visualización: El renderizado de los componentes gráficos es incorrecto o está mal optimizado.

Riesgos operativos

RO1 Documentación deficiente: La documentación no permite mantener ni ampliar el proyecto a futuro.

RO2 Problemas personales: Falta de motivación o problemas de salud física o mental se presentan durante el desarrollo.

RO3 Curva de aprendizaje elevada: Se encuentra una curva de aprendizaje elevada y se carece de experiencias con la tecnología a utilizar.

Riesgos de requerimientos

RR1 Requerimientos ambiguos: La definición incompleta o ambigua de los requerimientos puede provocar cambios en etapas avanzadas del desarrollo.

Riesgos de estimación

RE1 Subestimación del tiempo de desarrollo: La complejidad de enlazar diferentes tecnologías puede extender los plazos planeados.

Riesgos de alcance

RA1 Limitación del alcance funcional: Las restricciones de tiempo pueden dar pie a limitar el número de funcionalidades requeridas.

RA2 Carga de trabajo imprevista: El poder de cómputo resulta insuficiente, por lo que se reduce la complejidad de las simulaciones.